University of Colombo School of Computing

SCS 3311 — Mobile Application Design and Development

Mini-Project

HomeSense

Smart Home Monitoring and Control System

Demonstration Script

Group Members

Name	Index number	Module
R.L. Weerasinghe	23002204	Synchronisation, simulator and safety worker
W.T. Mahagamage	23001038	Device profiles, control interface and reporting
D.M. Isakya	23000643	Floor representation and grid mapping

August 2026

Purpose and constraints

The recording must not exceed **25 minutes**. This script targets **22 minutes**; the margin is deliberate, since live demonstrations generally run over.

The specification requires all three members to appear, each introducing themselves and stating their contribution. Each block therefore opens with that introduction rather than covering all three at the start.

Block	Member	Topic
1	D.M. Isakya (23000643)	Floor representation and device placement
2	W.T. Mahagamage (23001038)	Device profiles, control interface and reporting
3	R.L. Weerasinghe (23002204)	Synchronisation, simulator and safety worker

Preparation checklist

- ☐ Worker running (`npm start`), with the terminal visible in the frame
- ☐ Simulator open in a browser, worker status indicator showing online
- ☐ `npm run seed` already executed, so the reporting screen has history
- ☐ Phone screen mirrored, notifications enabled, do-not-disturb off
- ☐ Iron's `max_on_duration` set to 30 seconds
- ☐ A browser tab with `database.rules.json` open for section 6
- ☐ Screen recorder capturing both the phone and the desktop in one frame

1 Introduction — D.M. Isakya 0:00 – 1:30

Introduce yourself with your index number and state your module: floor representation, the grid, and device placement.

State the problem briefly: a house has several floors, appliances of different kinds, and some of them present a fire risk if left running.

Then state the central design decision once, clearly:

> Every switch in the system stores three separate values: the state the application requests, the state the hardware reports, and the status the server derives from the two. That separation is why the error and disconnected states in this system represent actual observations. The difference will be visible later in the demonstration.

2 Floors and grid — D.M. Isakya 1:30 – 5:30

Time	Action	Point to make
1:30	Floors list showing two storeys with live device counts	Multiple floor plans, each with its own grid
2:15	Open the ground floor	Plans are drawn as geometry rather than images, so they stay sharp at any density and the aspect ratio is exact
2:45	Indicate the legend	Shape encodes device kind, fill pattern encodes status. Nothing relies on colour alone
3:15	Tap an empty cell and place an outlet	Positions are stored as grid cells, not pixels, so a plan renders identically on a phone and a tablet
4:00	Rotate the phone	Placement is unaffected. The coordinate mapping is one pure class with sixteen unit tests
4:30	Add a floor, selecting a plan and grid size	Floors are added and managed at runtime
5:00	Open the gang box	One device, one cell, one heartbeat, with three independently addressable switches. The specification requires a single entity and the data model permits nothing else

Hand over to W.T. Mahagamage.

3 Device profiles — W.T. Mahagamage 5:30 – 10:00

Introduce yourself with your index number: device profiles, the control interface and usage reporting.

Time	Action	Point to make
6:00	Outlet card, toggle it	The simulator lamp responds within a second, with no refresh anywhere
6:45	Gang box, toggle switch 2 only	Independently addressable; switches 1 and 3 are unaffected
7:15	Master toggle	Addresses each slot in sequence, still as one device
7:45	Schedule editor on the porch light	On at 18:30, off at 06:00 — a window crossing midnight, which is the usual case
8:30	Safety editor on the iron	`max_on_duration`, using the specification's own field name. Enforced by the server, not by this screen
9:00	Switch the iron on; the countdown appears	The indicator drains as the permitted time is consumed and changes colour below twenty per cent
9:30	Camera tile, then full screen	Simulated snapshot and stream, as the specification permits. The configured URIs are shown beneath

Switch the iron off before continuing; the cut-off is demonstrated separately at 14:00. Hand over to R.L. Weerasinghe.

4 Simulator and fault injection — R.L. Weerasinghe 10:00 – 14:00

Introduce yourself with your index number: synchronisation, the hardware simulator and the safety worker. Keep the phone and browser both in frame throughout this section.

Time	Action	Point to make
10:30	Show the simulator	This represents the hardware. It writes the reported state and a heartbeat, and nothing else. It does not write status
11:00	Physical toggle on the ceiling light	A change originating at the wall switch. The phone was not touched, and the change appears immediately, recorded with source `SIMULATOR`
11:45	Simulate fault on the fan, then toggle it from the app	The relay stops responding. Nothing writes an error state
12:15	Wait approximately ten seconds and observe the app	The client reverts its optimistic switch at four seconds and warns; the worker publishes `ERROR` at ten seconds. The error was derived by a third process from a disagreement between two others
13:00	Unplug the kitchen outlet	The heartbeat stops. Within fifteen seconds the worker publishes `DISCONNECTED`

Time	Action	Point to make
13:30	Close the simulator tab entirely	The `onDisconnect()` handler fires and all devices become disconnected. The absence was recorded by the server

Reopen the tab and clear the fault before the next section.

5 Safety cut-off — R.L. Weerasinghe 14:00 – 17:00

This is the central demonstration.

Time	Action	Point to make
14:00	Set the iron to `max_on_duration = 30` and switch it on	The countdown begins and the simulator's heat bar fills
14:30	Force-stop the application, showing the settings screen	The phone is now out of the system entirely
14:45	Indicate the worker terminal and the simulator	Only these two processes are running
15:00	Wait, describing the countdown	No timer is held in memory. The worker recomputes elapsed time from the stored `onSince` value at every pass
15:30	The cut-off fires: the lamp goes out, a log line appears, a notification arrives	The appliance was switched off with the application force-stopped
16:00	Reopen the app and show the alert centre	The critical alert was written by the server. A client may only mark it acknowledged
16:30	Stop and restart the worker mid-countdown, repeating briefly	A restart loses nothing, because the state is in the database. The next pass acts immediately. This case is covered by a unit test

6 Enforcement and reporting 17:00 – 19:00

Time	Member	Action
17:00	R.L. Weerasinghe	Show `database.rules.json`. A write grant propagates in Realtime Database and cannot be withdrawn at a lower level, but validation rules do restrict client writes, and the Admin SDK bypasses validation. `status` therefore carries a validator that accepts a write only when the value is unchanged

17:45	R.L. Weerasinghe	State the limitation directly: the client and simulator share anonymous authentication, so the rules cannot distinguish them. What no client can do is publish a status value, which is the property the safety argument depends on
18:00	W.T. Mahagamage	Usage screen. Every figure derives from logged transitions rather than from comparing snapshots, so the cut-off that occurred with the phone closed is included
18:30	W.T. Mahagamage	Leaderboard, cut-off count and energy estimate, labelled as an estimate because it assumes rated wattage throughout
18:50	W.T. Mahagamage	CSV export through the share sheet

7 Architecture and limitations 19:00 – 20:30

One point each:

- **D.M. Isakya** — MVVM structure: composable, ViewModel, repository, data source. No composable accesses Firebase directly.
- **R.L. Weerasinghe** — Why Realtime Database rather than Firestore: per-child listeners and `onDisconnect()`. Why a standalone worker rather than Cloud Functions: the billing requirement, and the fact that the rules are pure functions that would transfer unchanged.
- **W.T. Mahagamage** — What was not done: a single home, simulated cameras, estimated energy figures, and the dependency on the worker running, which is why the simulator displays the worker's heartbeat.

8 Verification — W.T. Mahagamage 20:30 – 21:30

Run both suites on camera:

```
./gradlew test          # 73 tests
cd worker && npm test   # 40 tests
```

Highlight two cases: the cut-off fires at exactly `max_on_duration`, and it still fires correctly after a simulated worker restart.

9 Closing — D.M. Isakya 21:30 – 22:00

One sentence each on further work: custom authentication claims to separate client from hardware, support for multiple homes, and real camera streams.

Priority sequences

If time is short, these three sequences take precedence over the commentary.

1. **14:00 – 16:00** — the cut-off firing with the application force-stopped.
2. **11:00** — a change originating at the hardware appearing on the phone.
3. **12:15** — an error state derived from a sustained disagreement.

Contingencies

Problem	Response
Network unavailable	Use the demo build. It runs the same interface against an in-memory backend with a working cut-off, and the fault-injection controls are in the device sheet
Worker will not start	Check `worker/.env` and the service account key. The simulator's status indicator reports the condition
Notification does not arrive	Say so and continue. The cut-off has already occurred and the alert centre confirms it, which is itself the point being made
No data appears in the app	Confirm that `homeId` matches across the application, the worker's environment file and the simulator configuration